1 performance and delay

transmission medium

transmission medium the means by which information travels from a sender to a receiver
data encoding applying a specific code to data so it takes an equivalent form the medium carries; it decides how much data each signal state carries
media copper twisted pairs carry electrical signals; optical fiber and free space carry light; radio covers mobile and wireless links
medium determines bandwidth comes from the medium type, propagation delay from the link length; the third property is error rate, seen as packet loss

bandwidth

bandwidth number of bits a link carries per unit time, in bits per second (bps); it is the width of the pipe whose length is the propagation delay
bit rate units 1 Bps is 8 bps; networking prefixes step by powers of 1000, so 1 Kbps is 1000 bps, 1 Mbps is 1000 Kbps, 1 Gbps is 1000 Mbps

packet loss

packet loss a packet that never arrives is lost; how often this happens follows from the medium's error rate, not from the data
loss handling a transport protocol such as TCP has to detect what was lost and retransmit it without knowing why it went missing
random loss causes a solar flare, a nearby microwave oven, a cable plugged in wrong, and several transmitters sending at once

network delay

transmission delay $t_{\text{trans}} = \frac{N}{R}$ for a packet of N bits on a link of rate R, independent of distance; 1500 B is 12000 bits, so 12 ms at 1 Mbps and 12 microseconds at 1 Gbps
propagation delay $t_{\text{prop}} = \frac{d}{v}$ for a link of length d where signals travel at v; a medium's velocity factor gives its slowdown. Fiber at two thirds of light (200000 km/s): 3000 km takes 15 ms, 10000 km takes 50 ms
queueing delay $\frac{B}{R}$ for a packet arriving behind B bits of queued traffic on a link of rate R; a burst queues behind itself, the wait climbs with load, and a full queue drops arriving packets
packet delay $\left(\frac{N}{R}\right) + L$ with no queue in the way, where L is the link latency, the propagation delay; add the queueing delay when the packet waits. A 1500 B segment on a 1 Gbps link with 2 ms latency takes 2.012 ms
geostationary satellite about 35786 km above the equator; light takes about 120 ms each way, so the two crossings cost about 240 ms. ITU Recommendation G.114 (2003) uses 150 ms one-way for general network planning
which term dominates 1500 B on a 40 Gbps link with 10 microseconds of latency: 0.3 plus 10, so 10.3 microseconds. The same data on a 32 kbps link with 8 s of latency: 0.375 plus 8, so 8.375 s
Ethernet frame a 1500-byte frame is a 1460-byte segment plus a 20-byte TCP header and a 20-byte IPv4 header

network

network two or more endpoints joined by a communication medium; joining two or more networks gives an internetwork, of which the Internet is the largest

2 the web

HTTP

request and response format request: request line (method, path, version) + headers + optional body; response: status line (version, code, phrase) + headers + optional body; blank CRLF separates headers from body
HTTP 1.0 cost one new TCP connection per object; n small objects cost $3n$ RTTs, since one RTT opens the connection and one carries the request
HTTP 1.1 persistence persistent by default, either side closes when done; handshake paid once per connection, so n small objects cost $n + 2$ RTTs
pipelining (HTTP 1.1) write several requests on one connection without waiting; server replies in request order; the whole batch costs one round trip
HTTP/2 binary frames each naming a stream ID; many multiplexed streams on one TCP connection, so responses go out of order; priorities; HPACK header compression
HTTP/3 runs on QUIC over UDP, RFC 9000 in 2021; each stream is reliable on its own, so a lost packet blocks only that stream; keeps multiplexed streams and persistence
head-of-line blocking by version response queue ordered in 1.1 only; transport ordered in 2.0 only; 1.0 and 3.0 have neither

head-of-line blocking

Layer 7 against Layer 4 Layer 7: HTTP 1.1 pipelining keeps request order, so a slow backend call holds objects already fetched; Layer 4: TCP hands bytes over in order, so one lost packet stalls every HTTP/2 stream
fixes separate connections (costs the network: each runs its own congestion control); HTTP/2 streams clear Layer 7; QUIC per-stream reliability clears Layer 4

video on demand

on-demand video recorded in advance and played when the viewer wants it (films, YouTube, TikTok); nothing is ever late, so delivery runs ahead of playback and the client holds a buffer
real-time video active communication such as Skype or Zoom, played as soon as it can arrive; no buffer, since nothing was fetched ahead of time

video bitrate

video bitrate bits the file stores per second of playback; the client plays out of a buffer that only the network fills, so the bitrate is also the delivery rate the network must sustain
bitrate against network capacity above what the network delivers, the buffer drains faster than it fills and playback stalls; below it, capacity goes unused and quality is worse than needed
variable bitrate encoding one target rate is an average over the whole video, too low for an action scene and too high for a sleeping one; per-segment rates give each segment its own

progressive download

client buffer the server sends data into the buffer at rate C and the client reads and plays from it at rate R; the difference accumulates, so $C < R$ drains it and playback stalls once occupancy hits zero before the end f
occupancy curves $A(t) = Ct + b$ is the data delivered by time t and $S(t) = Rt$ the data shown by then, so occupancy is $A(t) - S(t)$, emptied by design exactly at f; b is the occupancy when playback starts, fetched in the k seconds before
worked case 3 GB at $R = 8$ Mbps over $C = 6$ Mbps: $f = 3000$ s, $b = (R - C)f = 6000$ Mb which is 750 MB, priming $k = \frac{b}{C} = 1000$ s, about 16.67 min
why the capacity varies new users joining or leaving, link failures and re-routes, and WiFi interference; the buffer runs playback at the average available bandwidth, so a high-variance client needs a larger one

3 video streaming

adaptive bitrate streaming (ABR)

ABR varies video quality to match what the network can carry; the algorithm runs throughout playback, not once at the start
ideal MBR selection pick the largest send rate C on offer with $C < N$, keeping playback rate R under capacity N; the client cannot measure N
ABR function bitrate $= f(x, y, z, \ldots)$; five obvious arguments: average past capacity, loss rate, latency, change in latency, bandwidth variance, none measurable
measurable substitute the client reads its own buffer occupancy; one target occupancy plus a rule turns it into a bitrate

media segmentation and HTTP chunking

media segmentation cut a video into chunks and fetch each as an ordinary HTTP object, in order, so the browser plays them back to back
HTTP advantages playback moves into the browser; servers stay stateless; chunks go into caches and CDNs, which is the scalability; send rate C can change per chunk

DASH (Dynamic Adaptive Streaming over HTTP)

DASH streams a video as ordinary HTTP requests for ordinary files; one quality level is one bitrate plus one resolution; nothing in the protocol says which level to ask for
MPD manifest XML media presentation description; AdaptationSet holds one stream (video or audio), SegmentTemplate gives the rule for finding a segment, Representation is one quality level
selection and audio the ABR algorithm on the client chooses the segments; audio is one stereo Representation at 128 kbps and 48000 Hz, fetched and decoded separately from the video

buffer-based rate adaptation (BBA)

BBA chooses each chunk's bitrate from the client buffer occupancy alone; playback drains the buffer at a steady rate, so unless the network replaces what is played the occupancy falls to a stall; BBA-0 is the simplest version
three regions lower reservoir of size r up from an empty buffer, then the cushion of size c_{u}, then the upper reservoir above $r + c_{\text{u}}$; rate is $R_{\min}$ through the lower reservoir, rises across the cushion, and is $R_{\max}$ through the upper
BBA-0 selection discrete ladder $R_{\min}$, R_1, R_2, R_3, $R_{\max}$; occupancy at or below r gives $R_{\min}$, at or above $r + c_{\text{u}}$ gives $R_{\max}$, else switch only when the curve crosses a neighbouring bitrate boundary, otherwise keep the previous rate
Puffer study 2020 BBA stayed among the best of the algorithms compared on 244,028 streams; roughly 0.23 percent stalled and 16.5 dB SSIM on all sessions, roughly 2.8 percent stalled and 14.5 dB on paths below 6 Mbit/s

worse is better

worse is better tight integration, a player and media server keeping RTSP session state at every step, lost to weak integration, a file server handing out stateless HTTP requests; the winner is backwards-compatible, less complex and easy to scale

4 transport layer, UDP, stop-and-wait

transport layer role

transport layer adds addressing of application processes instead of hosts, plus optionally reliable, in-order, paced delivery; it runs on the two end hosts only, and routers in between stop at the network layer
network layer promise best-effort only: a packet may be corrupted, delayed, dropped, reordered, or duplicated, and no rate guidance is given

ports and multiplexing

multiplexing and demultiplexing demultiplexing hands each arriving packet to the process that owns it, multiplexing marks each outgoing packet for its sender, both by port number
port ranges 16-bit field gives $2^{16} = 65536$ ports: well-known 0 to 1023, registered 1024 to 49151, dynamic 49152 to 65535

service model

services TCP gives reliable in-order delivery plus pacing but guarantees neither delay nor bandwidth; UDP gives neither; the receiver advertises how many bytes it can still take, and a window of zero stops the sender until room appears

user datagram protocol

UDP provides multiplexing and demultiplexing for application processes and nothing else: no handshake, no delivery guarantee, no ordering, no pacing; RFC 768 defined it in 1980
header exactly 8 bytes, four 16-bit fields in fixed order: source port at octets 0 to 1, destination port at 2 to 3, length at 4 to 5, checksum at 6 to 7, then the data
length field and maximum sizes length covers header and data, so its smallest value is 8 and a 100-byte payload reads 108; the whole datagram is at most $2^{16} - 1 = 65535$ bytes, IPv4 leaves at most $65535 - 20 - 8 = 65507$ bytes of data, and IPv6 leaves $65535 - 8 = 65527$
byte order every 16-bit field is big-endian, the more significant byte first, which the standards call network byte order; port 1234 is 0x04D2 and travels as 04 then D2
checksum the one's complement of the one's complement sum of the 16-bit words of the datagram; a field of zero means the receiver skips the check, optional in IPv4 but required in IPv6, and the pseudo-header it also covers is 12 bytes in IPv4 and 40 bytes in IPv6, holding the addresses, protocol number 17 and the length
named users DNS to port 53, DHCP in four datagrams (discover, offer, request, acknowledge) with the offer returning to port 68, NTP to port 123, and real-time VoIP at 20 ms of audio per packet, where a loss is covered from nearby samples rather than retransmitted

stop-and-wait ARQ

rule send one packet and wait for its acknowledgement before sending the next; a timer covers the wait and its expiry is the evidence that the packet or the acknowledgement went missing
sender and receiver rules the sender numbers the packet, transmits it and starts a timer, moves on when the acknowledgement arrives, and resends on expiry; the receiver acknowledges every packet it accepts, ignores a corrupted one, and discards a repeated sequence number while acknowledging it again, so the application sees the data once
four outcomes the frame and its acknowledgement arrive, the frame is lost, the acknowledgement is lost, or the acknowledgement arrives after the timer fired; the sender cannot tell a lost frame from a lost or late acknowledgement, since in both no acknowledgement arrived in time
timer choice and cost below the round trip it fires on a path that lost nothing and every frame is carried twice; far above it the sender sits idle after a loss; a loss costs the timeout plus another round trip, so a 250 ms timeout on a 200 ms path costs 450 ms
sequence numbers one bit suffices, alternating between 0 and 1 and flipping on every successful transfer, so a repeat of the bit being waited for is a duplicate; reassembly could rest on a convention, but a retransmission cannot
throughput one packet per round trip, $\frac{\text{MSS}}{\text{RTT}}$; on 50 Mbps with a 200 ms round trip and 1460-byte packets that is $\frac{1460}{0.2} = 7300$ bytes per second, about 0.12% of the link, needing about 856 packets in flight to fill it, while a window of 8 packets reaches 58400 bytes per second

5 sliding window and ARQ

sliding window protocol

core idea a sender keeps several unacknowledged segments in flight: it sends every segment inside a window and slides the window forward as acknowledgements arrive
what bounds the window the advertised window from the receiver and the congestion window from the network
acknowledgement number names the next byte the receiver expects and is cumulative; the window covers the segments sent but not yet acknowledged, and its left edge moves up to the acknowledged byte

acknowledgement

cumulative acknowledgement names the next byte the receiver expects, so 4000 reports that bytes 0 to 3999 arrived; a lost report costs nothing since the next number covers more, and a repeat means the receiver still waits there
cost of a gap the count stops at the first missing byte, so one loss costs every segment sent after it; a 100 KB page is 102400 bytes, about 70 segments of 1460 bytes
selective acknowledgement keeps the cumulative number and adds the ranges held past the gap, carried in the options field of the header; one loss then costs 1 segment instead of a window; RFC 2018 defines it

go-back-N ARQ

recovery rule when the timer for packet k expires, the sender resends packet k and every packet after it in the window
receiver rule accepts only the next expected packet and discards everything else, so it holds no out-of-order data and one buffer slot is enough; it always acknowledges the next expected number
sender in flight up to N unacknowledged packets with window size N and one timer per packet; a cumulative acknowledgement of n means every packet before n arrived in order and retires many timers at once
window must cover a round trip the window must cover one round trip or the sender runs dry; 50 Mbps with a 200 ms round trip gives a bandwidth-delay product of 1.25 MB, about 856 packets of 1460 bytes
partial acknowledgement impossible here: the receiver never holds a later packet while an earlier one is missing, so its number always names the first missing packet, and that case belongs to selective repeat
timer rules equal timers: the oldest expires first and its recovery covers the rest, so one timer for the oldest packet behaves the same; a retransmitted packet gives no round-trip sample, by the Karn rule

selective repeat ARQ

core idea keeps the sliding window of go-back-N but lets the receiver hold what arrives early, so only the lost packet is resent and one loss costs one retransmission instead of a window
partial acknowledgement the receiver acknowledges every packet on arrival, early ones included, and the number names only that packet; the window stops at the lowest missing packet, sliding only over an acknowledged left edge
receiver buffer it stores out-of-order packets rather than discarding them and acknowledges each on arrival; the buffer is one slot per window position, 8 times 1460 is 11680 bytes, against the one packet go-back-N keeps
sequence number space k bits give 2^k values and the window covers at most half, 2^{k-1}, since the receiver may still hold a packet whose number a new packet reuses; 3 bits allow a window of 4, and a window of 8 needs 4 bits

6 TCP basics

TCP service and windows

TCP service A reliable, ordered byte stream over an unreliable best-effort network; the bytes carry no message boundaries
Maximum segment size Limits the data in one segment, commonly 1460 bytes, so header and data still fit the network layer limit
Two send windows The receiver advertised window and the congestion window; the smaller decides how many bytes may be in flight

Header

Header length Fixed part 20 bytes, options 0 to 40 bytes padded to a multiple of 4, whole header at most 60 bytes
Header field offsets Ports 0 to 3, sequence number 4 to 7, acknowledgement number 8 to 11, data offset and reserved 12, flags 13, window 14 to 15, checksum 16 to 17, urgent pointer 18 to 19
Data offset Counts 32-bit words from the segment start to the data; 5 means 20 bytes and 15 means 60 bytes
Big-endian fields Every multi-byte field travels network byte order, most significant byte first; port 80 is 0x0050 sent as 00 then 50, read the other way it would say 0x5000, which is 20480

Flags

Eight flags Octet 13, most significant first: CWR 0x80, ECE 0x40, URG 0x20, ACK 0x10, PSH 0x08, RST 0x04, SYN 0x02, FIN 0x01
Flag combinations A SYN that acknowledges reads 0x12, ACK alone 0x10, and a FIN that acknowledges 0x11

Sequence numbers

Sequence numbering Numbers count bytes: a segment holds the number of its first byte, and an acknowledgement names the next byte expected, $S + L$
Numbers consumed Data takes one per byte, a SYN takes one, a FIN takes one; ACK, PSH, URG, RST, CWR and ECE take none
Numbering example Start at 1000, send 500 bytes, close: SYN 1000, data 1001 to 1500, FIN 1501, last acknowledgement 1502
Sequence space The 32-bit field covers $2^{32} = 4,294,967,296$ bytes, a little over 4 GB; at 10 Gbit/s a stream that never stops wraps in about 3.4 seconds
Serial number arithmetic Compares the modular difference, so the number after $2^{32} - 1$ is 0 and a difference below 2^{31} counts as forward; the window stays below 2^{31}
Initial sequence number Chosen fresh rather than zero, so segments left over from a reused address and port pair fall outside the new connection window

Connection setup

Three-way handshake SYN seq x; SYN and ACK seq y ack $x + 1$; ACK seq $x + 1$ ack $y + 1$
Why three segments Each side must send its own initial number and see it acknowledged, and only the third segment acknowledges the server number
SYN retransmission timer 3 seconds by default, 6 in some implementations, since no round-trip time has been measured; backoff gives $3 + 6 + 12 = 21$ seconds
Lost handshake segments A lost SYN is repeated with the same x; a lost final ACK draws the server retransmitted SYN and ACK

Connection teardown

Four-segment close Client FIN (c, s); server ACK $(s, c + 1)$; server FIN $(s, c + 1)$; client ACK $(c + 1, s + 1)$
Close states The closing client passes FIN_WAIT_1, FIN_WAIT_2, TIME_WAIT; the server passes CLOSE_WAIT, LAST_ACK, CLOSED
Maximum segment lifetime The longest time a segment survives inside the network, fixed at 2 minutes as an engineering choice; no header field carries it
TIME_WAIT length The host that closes first waits $2 \times 120 = 240$ seconds; Linux and macOS keep it about 60 s and Windows 240 s by default

Simultaneous close Both hosts enter CLOSING after FIN_WAIT_1 and reach TIME_WAIT on the acknowledgement of their own FIN; CLOSE_WAIT and LAST_ACK never appear
Reset RST aborts the connection, is not acknowledged, takes no sequence number, and answers a segment for a connection the host does not have

Connection states

Eleven states CLOSED, LISTEN, SYN_SENT, SYN_RECEIVED, ESTABLISHED, FIN_WAIT_1, FIN_WAIT_2, CLOSE_WAIT, CLOSING, LAST_ACK, TIME_WAIT
Opening transitions Server CLOSED to LISTEN on listen, to SYN_RECEIVED on SYN, to ESTABLISHED on ACK; client CLOSED to SYN_SENT on connect, to ESTABLISHED on SYN and ACK

7 RTT, flow control, bandwidth-delay product

RTT and the retransmission timer

RTT time for a segment to reach the far end plus the time for its acknowledgement to return; the return pays propagation only, so a round trip is twice the propagation delay plus one transmission delay, not twice the one-way delay
EstimatedRTT $\text{EstimatedRTT} = (1-\alpha)\ \text{EstimatedRTT} + \alpha\ \text{SampleRTT}$, weight $\alpha = 0.125$
DevRTT $\text{DevRTT} = (1-\beta)\ \text{DevRTT} + \beta\ |\text{SampleRTT} - \text{EstimatedRTT}|$, weight $\beta = 0.25$
TimeoutInterval $\text{TimeoutInterval} = \text{EstimatedRTT} + 4\ \text{DevRTT}$; Request for Comments 6298 (2011) floors it at 1 second
Sample rules one sample runs from sending a segment to receiving its acknowledgement; retransmitted segments are left out, since their acknowledgement may answer either copy
Worked timer estimate 102.5 ms with deviation 12.5 ms gives 102.5 + 4 times 12.5, so 152.5 ms

Flow control and the receive window

Flow control keeps a fast sender within the buffer room the receiver has left; without it the receiver buffer overflows
Receive window (advertised window) buffer room the receiver has left, published in the headers the receiver sends back; set by buffer size, CPU processing ability, and system memory size
Sending window min(RWND, CWND); the sender re-reads the advertised window from every reply and lowers its window at once when the window shrinks
Worked buffer a 64 MSS buffer holding 30 MSS advertises the remaining 34 MSS, and the value changes as the application reads data

Bandwidth-delay product

Bandwidth-delay product path rate times its round-trip time, the data that must be in flight to keep the path busy; a window below it leaves the link partly idle, above it builds queues that overflow into loss
Why the round trip the sender holds its bytes until the acknowledgement returns, so the one-way delay counts only the outbound half. With 100 ms of propagation each way and 0.23 ms of push the round trip is 200.23 ms, so 50 Mbps needs 1.25 MB against 626 KB one way; twice the one-way delay gives 1.253 MB, 0.23 ms too much
Rate from the window $\frac{c}{\text{RTT}}$ bytes per second for c bytes in flight, so full rate R needs $c = R \times \text{RTT}$
Worked sizes 1 Gbps over a 250 ms round trip holds 31.25 MB, about 21400 segments of 1460 bytes; 10 Mbps over a 50 ms round trip needs 62.5 KB

8 congestion control

congestion control

congestion control the rule that decides how large a window a sender may keep in flight on a path whose bandwidth and round trip time nothing reports
the only feedback a drop means the sender is faster than the path carries, a long stretch without a drop means it is below capacity, and a timeout is harsher evidence than three duplicate acknowledgements because it suggests many packets lost at once
AIMD rules additive increase adds a fixed step, one maximum segment size per round trip; multiplicative decrease halves the window on a loss
Chiu and Jain properties efficiency, fairness, convergence, and distributedness; of the four increase and decrease combinations only additive increase with multiplicative decrease gives all four
AIMD convergence senders holding 60 and 40 packets on a 100 packet link each add 1 per round trip, reach 80 and 60 and overflow, then each loss halves both windows, so the gap of 20 packets falls by half per loss toward 50 packets each
algorithm landscape Reno and NewReno are the classics, CUBIC is the default on Linux and macOS, and BBR replaces the loss signal with a measurement of bottleneck bandwidth and minimum round trip

congestion collapse

congestion collapse useful throughput falls far below what the links could carry while the links stay busy, because retransmissions take the capacity fresh data needed; ordinary congestion costs delay and the data still arrives late, collapse costs the data itself
the 1986 collapse the National Science Foundation network in October 1986 fell from 32 kbit/s to 40 bit/s, a factor of 800, while the links ran at full load
before congestion control the only limit on a sender was the receiver advertised window, which is flow control, so a drop made each sender resend a whole window and the copies caused more drops
the repair Van Jacobson put it in the end hosts: the window became an estimate of the path capacity, shrinking on loss and growing without it, with no change to routers or applications, as set out in his 1988 paper “Congestion Avoidance and Control”
repaired rule on a loss resend only the missing segment and halve the window, so a window of 20 segments becomes 10 and the copies on the link fall from a whole window to half of one

TCP congestion control

congestion window CWND is the sender estimate of how much it may have outstanding and ssthresh marks where the window stops doubling; the sending window is min(RWND, CWND), with RWND the receiver side bound, an initial CWND of 4 MSS, and a default ssthresh of 64 KB
slow start the window starts at 4 maximum segment sizes and adds one per acknowledgement, so it doubles once per round trip, giving 4×2^n after n round trips; once it reaches ssthresh the growth becomes linear
congestion avoidance each acknowledgement adds $\frac{\text{MSS}}{\text{CWND}}$, about one maximum segment size per round trip, and the window keeps growing while no loss is reported
fast recovery third duplicate acknowledgement: $\text{ssthresh} = \frac{\text{CWND}}{2}$ and $\text{CWND} = \text{ssthresh} + 3$ MSS; the new acknowledgement sets CWND = ssthresh and returns to congestion avoidance, while a timeout sets CWND = 1 MSS instead
sawtooth slope the linear climb gains one maximum segment size per round trip, so a 10 ms path gains 100 MSS per second and a 100 ms path gains 10; Reno flows with round trips of 10 ms and 50 ms settle at 5/6 and 1/6 of the link
Reno limits a large bandwidth-delay product takes too long to fill, a wireless loss is misread as congestion, and the bottleneck queue stays full; CUBIC, Compound, and HighSpeed attack the first, BBR and Copa the queue, and RED, DCTCP, and TIMELY move the decision into the network

9 loss recovery, Mathis, CUBIC, fairness

loss recovery

two loss signals retransmission timer expiry, and three duplicate acknowledgements
duplicate acknowledgement receiver gets a segment out of order and re-sends an ACK naming the same next-expected byte
duplicate ACK count 1 or 2 can be ordinary reordering; the third means the segment is lost, not late
fast retransmit on the third duplicate ACK the sender retransmits the missing segment at once, without waiting for the timer
timeout retransmission timer started when the segment is sent; on expiry resend the oldest unacknowledged segment
why the timer is still needed a loss with no later segment behind it produces no duplicate ACKs
timeout cost also drops the congestion window to its initial value, undoing more than a fast retransmit

Mathis equation

Mathis equation throughput $= \frac{\text{MSS}}{\text{RTT}} \cdot \frac{C}{\sqrt{p}}$, the steady-state rate of a TCP Reno connection in congestion avoidance
terms MSS maximum segment size, RTT round-trip time, p loss rate, C a constant fixed by how the protocol reacts to loss
square root effect throughput falls only as $\frac{1}{\sqrt{p}}$, so cutting losses tenfold buys about three times the throughput
average window after a loss at window W, congestion avoidance climbs from $\frac{W}{2}$ back to W, so the average in flight is $3\frac{W}{4}$
average throughput $\frac{3}{4} \cdot \frac{W}{\text{RTT}}$; $W = 20$ segments of 1500 B over a 0.1 s round trip gives 225000 B per second
high bandwidth need 10 Gbps over a 100 ms round trip needs 83333 segments of 1500 B in flight, so the equation demands $p \approx 2.1 \times 10^{-10}$

CUBIC TCP

idea keeps the loss signal of TCP Reno but replaces linear growth with a cubic curve in elapsed time, so flows of different round trips grow alike
window function $W_{\text{cubic}} = C(t-K)^3 + W_{\max}$, with C a scaling factor, t the time since the last reduction, $W_{\max}$ the window at that reduction
K $K = \sqrt[3]{W_{\max}\frac{\beta}{C}}$ is the time the curve takes to return to $W_{\max}$, with β the multiplicative decrease factor
worked curve with $W_{\max} = 100$ segments, $\beta = 0.8$, $C = 0.4$: $K = \sqrt[3]{200} \approx 5.85$ s, window 180 segments at $2K$ and 740 at $3K$
loss response cut the window by 20% at a loss, deepened to 40% when the estimate of the recent maximum shrinks because the path itself got smaller
repeated losses from a window of 100 segments, two losses leave 64 segments under CUBIC TCP, against 25 under TCP Reno
long fat path Reno needs 83253 round trips, about 2.3 hours, to reach 83333 segments; CUBIC covers it in about 65 s, about 128 times faster

fairness measure

fairness index $F(x) = \frac{\left(\sum_i x_i\right)^2}{n\sum_j x_j^2}$ over shares $x_1, \ldots, x_n$: 1 for equal shares, falling toward the floor $\frac{1}{n}$
variance form $F = \frac{1}{1+\frac{\text{Var}[X]}{E[X]^2}} = \frac{1}{1+\text{CV}^2}$, with $\text{CV} = \frac{\sqrt{\text{Var}[X]}}{E[X]}$; it equals 1 exactly when the shares are equal

10 queueing and fair sharing

max-min fairness

max-min fairness Maximise the smallest share: no flow’s share can be raised without lowering the share of a flow that already has less; a flow gets only what it demands and the surplus goes to flows wanting more
mouse, elephant, and demand A mouse wants little and never uses more, an elephant would take the whole link; flow demand r_i is the most it would use. Splitting 1 Gbps equally between a 40 Mbps stream and a download gives 500 Mbps each, wasting $1000 - (40 + 500) = 460$ Mbps
allocation rule and water filling $a_i = \min(f, r_i)$ with f the level at which the shares sum to capacity C; water filling reaches it by filling every bucket at the same rate, stopping each when full and continuing with the rest
work conservation A scheduler is work-conserving when it never leaves the resource idle while a job waits; the level rises until capacity is used, so no bandwidth goes unspent while a flow wants more

fair queuing

fair queuing design One queue per flow, served in turn and work-conserving, approximating the ideal of generalized processor sharing where every flow is served at once at a fixed fraction of the link; unlike a first-come first-served queue, which serves in arrival order
turn duration and packet size A packet of L bits occupies a link of rate R for $\frac{L}{R}$ seconds: 1500 bytes is 12000 bits, 120μs at 100 Mbps; round robin gives 1500-byte and 500-byte flows equal turns, so the larger takes $\frac{12000}{4000} = 3$ times the bits
virtual clock and reserved rate $V(t)$ advances at link rate while busy and stops when idle; $r_q = \frac{R}{n}$ for n equal flows, or $r_q = \frac{Rw_q}{\sum_q w_q}$ with weights, so weights 3 and 1 on 80 Mbps give 60 and 20 Mbps
tag rule $F_{q,i} = \max(V(t), F_{q,i-1}) + \frac{p_{q,i}}{r_q}$, where $p_{q,i}$ is the packet bits and $F_{q,i-1}$ the tag of the packet ahead in the same queue; the scheduler sends the smallest tag first
approximation error Transmission is not preemptive, so the order can be wrong until the packet on the wire finishes: at most one packet transmission time, 12μs for 1500 bytes at 1 Gbps

token bucket

token bucket Tokens arrive at rate r into a bucket holding at most b, every bit sent costs one token, and the sender cannot send while it has too few; the average is exactly r and the burst at most b, and an idle sender banks credit up to b
burst size Capacity b sets how far ahead of the average a sender may run; a very large bucket lets one flow fill the bottleneck and delay every flow behind it, while a small one smooths traffic that arrives in clumps. Refill from empty takes $\frac{b}{r}$, so 0.8 s at 1 Mbps for 100 kB

network calculus

network calculus Worst-case bounds on queue length, waiting time, and drain time from an arrival curve and a service curve, with no simulation and no traffic trace
arrival curve $A(t)$ bounds the data that may have arrived by time t; a token bucket of rate r and burst b gives $A(t) = rt + b$ in the worst case that the bucket starts full, so 10 Mbps with a 200 kB burst gives $A(t) = 10^7 t + 1600000$ bits
service curve and the gap $S(t)$ is the data the router can serve in time t, and a bare output link of rate r_2 gives $S(t) = r_2 t$; the vertical gap between $A(t)$ and $S(t)$ is the backlog, the horizontal gap the waiting time, and the curves meeting means the queue has drained
drain time and worst wait Solve $S(t) = A(t)$: $30t = 20t + 12$ empties at $t = 1.2$ s, where 20 and 12 come from adding the rates and bursts of two senders; a burst b over service rate r_2 waits at most $\frac{b}{r_2}$, so $\frac{1600000}{2\times10^7} = 0.08$ s

11 IP and addressing

IPv4 addresses

IPv4 format 32-bit address written as four decimal octets separated by dots, such as 128.2.42.10; each octet holds 8 bits running 0 to 255, so the space holds 2^{32} = 4,294,967,296 addresses.
classful allocation first octet 1 to 126 gives class A at default mask /8, 128 to 191 class B at /16, 192 to 223 class C at /24; block sizes are 2^{24} = 16,777,216, 2^{16} = 65,536 and 2^8 = 256.
special and private IPv4 224 to 239 is multicast (224.0.0.1 reaches all local-link hosts), 127 is loopback (127.0.0.1 is the host), 255.255.255.255 is local broadcast; private 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16 are never routed publicly.

masks and subnets

mask and prefix sizes the mask is a run of 1 bits then a run of 0 bits, never mixed; and-ing an address with its mask clears the host bits, so a prefix /n holds 2^{32-n} addresses with $2^{32-n} - 2$ usable: /24 gives 256 and 254, /26 gives 64 and 62.
forwarding decision a host ands its own address and the destination with its mask: equal results mean the destination is on-link and the frame goes straight to it, different results send it to the gateway; a router matching several prefixes uses the longest one, so /24 beats /16.

IPv6

IPv6 size and notation 128-bit address written as eight groups of four hexadecimal digits separated by colons, each group 16 bits; the space holds 2^{128} = about 3.4×10^{38} addresses, 2^{96} times the IPv4 space.
IPv6 shortening and prefixes drop leading zeros in each group, replace the longest run of all-zero groups with one double colon, write lowercase; ::1 is loopback and :: is unspecified; a /64 leaves 64 interface-identifier bits and fe80::/10 is link-local.

obtaining an address

DHCP a joining host asks the network for its address; the exchange is discover, offer, request, acknowledge, and the acknowledgement also carries the gateway and the address of a local name server.

anycast

anycast the same address is put on several machines and the network decides which one serves a packet; each copy advertises the same prefix, so a router forwards along the shortest path to the nearest instance.
routing table cost n copies of one address announce one shared prefix, so a router holds 1 entry for it with a path to each of the n copies, not n entries.
path choice a router chooses between the paths to the copies by its own metrics, usually the nearest; the choice is per packet, so packets of one connection may take different routes and reach different copies.
content delivery entry point a content delivery network announces one address from every city it serves, so a client opens a connection to the nearest point of presence and the network carries the request there; the client chooses no site.

anycast root servers

root servers each of the thirteen root server identities is replicated at many locations and the system runs more than a thousand instances; a-root-servers.net answers at 198.41.0.4 from every location hosting a copy, and a failed copy costs nothing.

anycast trade-offs

state and route changes a service reached by anycast must keep no state that only one instance holds, so shared state sits where every copy can reach it; a route change sends later packets of a connection to another copy.

12 domain name system

names and hierarchy

DNS maps human-readable names to Internet Protocol (IP) addresses; the earlier single hosts file failed scalability, high availability, correctness and fast lookups

zones and delegation

zone the part of the namespace one set of authoritative name servers answers for; a name is one node, while a zone is the set of nodes served together
delegation the parent zone holds NS records naming the child servers and nothing below the cut; the child operator serves the names underneath
glue the parent also holds A records for a child server whose name lies inside the child zone, breaking the circle where reaching the child needs that address
root zone the topmost zone, holding only delegations to top-level domains with glue and no host records, DNSSEC-signed with its key-signing key as the trust anchor of a validating resolver

root servers

root servers thirteen identities labelled A to M, each run by one organisation with many machines; twelve organisations run them, and one organisation runs two
root hints the bootstrap list of root server addresses compiled into resolver software, not learned from the Domain Name System itself

resolution

recursive versus iterative the client query to the local resolver is recursive, promising the final answer; the resolver queries down the tree are iterative, each server returning its best answer, usually a referral
the walk resolving www.ece.ust.hk asks a root server, then the hk servers, then the ust.hk servers, then the ece.ust.hk servers, which return 192.0.2.10

caching

time to live each cached answer carries a time to live, and the resolver serves the cached answer until it expires before asking the authoritative servers again
negative caching NXDOMAIN for a nonexistent name and an empty answer for a missing type are cached for the minimum field of the zone start of authority record

resource records

record fields owner name, time to live, class, type and data, with the type deciding what the data means
record types A: IPv4 address, AAAA: IPv6 address, NS: zone authoritative server, CNAME: alias, MX: mail exchanger with preference, PTR: reverse lookup name, TXT: free text, SOA: zone authority
CNAME rule a resolver follows an alias to the canonical name, and a name holding a CNAME cannot hold other records, so an alias carries no address of its own
SOA timers the SOA of ece.ust.hk carries serial 2026101201, refresh $\frac{7200}{3600} = 2$ h, retry $\frac{3600}{3600} = 1$ h, expire $\frac{1209600}{86400} = 14$ d and minimum 3600 s

message format

header the fixed part of a query or response header is 12 bytes and carries an identification, a set of flags, and a count for each of the four sections
sections and flags question, answer, authority and additional sections; flags report query or response, authoritative answer, recursion desired and recursion available